

GPU-Accelerated Edge-Preserving Image Denoising: A Shared-Memory Tiled CUDA Implementation of the Bilateral Filter

Mahad

Department of Computer Science

Mohammad Ali Jinnah University

Karachi, Sindh, Pakistan

Email: {author}@student.maju.edu.pk

Abstract—The bilateral filter is a non-linear, edge-preserving smoothing operator that suppresses noise in homogeneous regions while retaining sharp structural boundaries. Unlike a Gaussian blur, its weights are spatially variant and data-dependent, so the kernel is neither pre-computable nor separable; for a window radius r it costs $\mathcal{O}(r^2)$ exponential evaluations per pixel, making a serial CPU implementation prohibitively slow on high-resolution imagery. This work presents a CUDA implementation that maps the filter onto NVIDIA’s Single-Instruction-Multiple-Threads (SIMT) execution model. The design assigns one thread per output pixel using two-dimensional 16×16 thread blocks, pre-computes the invariant spatial Gaussian weights once on the host and broadcasts them from `__constant__` memory, and collaboratively stages each image tile—together with its r -pixel halo—into on-chip shared memory through a divergence-free one-dimensional strided load guarded by a `__syncthreads()` barrier. Boundary conditions are handled by clamped (edge-replicated) addressing rather than conditional branching. Against a single-threaded CPU baseline on identical inputs, the shared-memory kernel achieves a kernel-only speedup of roughly 3.6×10^2 to 4.6×10^2 , and an end-to-end (transfer-inclusive) speedup of up to $\approx 176\times$ at 4096×4096 resolution. Numerical output matches the serial baseline to within a maximum absolute error of 4.7×10^{-6} . A combined Amdahl/Gustafson analysis identifies the host-device PCIe transfer as the dominant serial component and quantifies the resulting speedup ceiling, motivating pinned-memory and stream-overlap optimizations as future work.

Index Terms—Bilateral filter, edge-preserving smoothing, image denoising, CUDA, GPU computing, SIMT, shared memory, memory coalescing, halo exchange, Amdahl’s law, parallel computing.

I. INTRODUCTION AND MOTIVATION

DIGITAL images acquired by physical sensors are invariably corrupted by noise—Gaussian, Poisson, salt-and-pepper, or speckle—that must be attenuated before any downstream perceptual or algorithmic use. The classical remedy is linear spatial filtering, predominantly the Gaussian blur, which is cheap and separable. Linear filters, however, are

spatially invariant: they convolve the same static kernel across the entire image and therefore attenuate *all* high frequencies indiscriminately, blurring exactly the sharp edges and contours that carry the most semantic information. In domains where geometric fidelity is paramount—medical magnetic-resonance imaging, computer vision, and autonomous optical navigation—this destruction of structure renders the smoothed image useless for subsequent analysis.

Tomasi and Manduchi resolved this tension with the *bilateral filter*, a non-linear operator that replaces each pixel with a weighted average of its neighbours, where the weight of a neighbour combines two terms: a *spatial* (domain) Gaussian that decays with geometric distance, and a *range* Gaussian that decays with photometric (intensity) difference [1]. When a neighbour lies across a strong edge, its large intensity difference drives the range weight toward zero, so the averaging is effectively halted at the boundary. Noise is suppressed inside homogeneous regions while edges remain crisp.

This selectivity carries a severe computational price. Because the range term depends on the actual local intensities, the bilateral kernel is *spatially variant*: it cannot be pre-computed as a static matrix, and it does not separate into independent one-dimensional passes the way a Gaussian does [3]. Consequently a window of radius r requires $(2r + 1)^2$ exponential evaluations *per pixel*, an $\mathcal{O}(Nr^2)$ workload for an N -pixel image. On a CPU this scales poorly: a multi-megapixel image incurs seconds of latency. Crucially, the per-pixel computations are mutually independent—an embarrassingly parallel structure that maps naturally onto the thousands of arithmetic units of a modern graphics processor. This independence, combined with the high arithmetic intensity of the transcendental range weights, makes the bilateral filter an ideal candidate for the SIMT model exposed by NVIDIA CUDA [9].

Objectives and Contributions

The scope of this project is a deliberately minimal but complete and rigorously analysed CUDA implementation.

Project report for CS3230 *Parallel & Distributed Computing*. Instructor: Engr. Abdul Basit. Source code is provided as a single self-contained translation unit (`main.cu`) with public-domain header-only image I/O.

Specifically, this report contributes:

- 1) a single-channel, floating-point bilateral filter kernel for the GPU that combines `__constant__`-memory broadcast of the invariant spatial weights with *shared-memory tiling* and halo staging to eliminate redundant global-memory traffic;
- 2) a divergence-free, one-dimensional *strided* collaborative load with clamped boundary addressing, avoiding the warp divergence that naive per-boundary conditionals would induce;
- 3) a serial CPU baseline used both as a ground-truth correctness oracle (validated to an epsilon tolerance) and as the reference for speedup; and
- 4) a quantitative performance study—spanning four image resolutions and three block configurations—interpreted through Amdahl’s and Gustafson’s laws, which isolates the host–device transfer as the binding bottleneck.

II. RELATED WORK AND BACKGROUND

The bilateral filter was introduced by Tomasi and Manduchi [1] as a combined domain–range generalisation of earlier non-linear smoothers; it remains the canonical formulation and is the one implemented here. Because its brute-force cost is prohibitive, a substantial literature targets *acceleration*. Durand and Dorsey reformulated the filter as a convolution in a higher-dimensional space and used down-sampling to obtain a fast approximation for high-dynamic-range tone mapping [2]. Paris and Durand generalised this into a signal-processing framework that bounds the approximation error and yields orders-of-magnitude speedups at the cost of exactness [3]. Chen, Paris, and Durand introduced the *bilateral grid*, a data structure enabling real-time edge-aware processing on graphics hardware [4]. Beyond imaging, Fleishman *et al.* adapted the same domain–range principle to denoise 3-D triangle meshes, demonstrating the generality of the formulation [5].

A second, orthogonal body of work concerns *how* to map stencil computations efficiently onto GPUs. Micikevicius’s finite-difference study established the shared-memory tiling-with-halo pattern that is now standard for neighbourhood operators on CUDA [8], and the NVIDIA best-practices literature codifies the memory-coalescing, occupancy, and constant-cache broadcast guidance that this implementation follows [10], [11].

Gap addressed: The approximate methods [2]–[4] trade numerical accuracy and implementation simplicity for speed, which is appropriate for production systems but obscures the underlying parallel-computing mechanics. The present work instead implements the *exact* brute-force filter but applies the full shared-memory tiling discipline of [8], and then subjects the result to a formal speedup/scalability analysis. The aim is a transparent, reproducible baseline whose every architectural decision—tile geometry, halo width, memory residency, synchronisation—can be tied directly to measured performance, rather than a black-box approximation. Table I situates the bilateral filter among common smoothers.

TABLE I: Comparison of common image-smoothing filters.

Filter	Paradigm	Edge pres.	Complexity
Mean	Linear average	Poor	$\mathcal{O}(r^2)$, separable
Gaussian	Linear weighting	Poor	$\mathcal{O}(r)$, separable
Median	Rank selection	Moderate	Sorting per pixel
Bilateral	Non-linear dual weight	Excellent	$\mathcal{O}(r^2)$, <i>non-separable</i>

III. METHODOLOGY AND PARALLEL DESIGN

A. Mathematical Formulation

Let I_p denote the intensity of the target pixel at coordinate p and I_q that of a neighbour q within the window S of radius r . The filtered output is

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in S} G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(|I_p - I_q|) I_q, \quad (1)$$

where the normalisation factor preserves total luminance,

$$W_p = \sum_{q \in S} G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(|I_p - I_q|), \quad (2)$$

and the two Gaussian terms are

$$G_{\sigma_s}(\|p - q\|) = \exp\left(-\frac{\|p - q\|^2}{2\sigma_s^2}\right), \quad (3)$$

$$G_{\sigma_r}(|I_p - I_q|) = \exp\left(-\frac{(I_p - I_q)^2}{2\sigma_r^2}\right). \quad (4)$$

The spatial parameter σ_s controls how far the filter reaches; the range parameter σ_r sets the intensity-difference threshold above which a gradient is treated as an edge. As $\sigma_r \rightarrow \infty$ the range term becomes uniform and the operator degenerates into a plain Gaussian blur.

The decisive observation for parallelisation is that (3) depends *only* on the fixed integer offset (dx, dy) between a pixel and its neighbour, never on intensity. The spatial kernel is therefore an $(2r + 1) \times (2r + 1)$ *constant* matrix that can be computed once. By contrast (4) is intensity-dependent and must be re-evaluated for every neighbour of every pixel—this is the irreducible parallel workload.

B. Serial Baseline

The CPU reference is a direct transcription of (1): four nested loops (two over pixels, two over the window) accumulate $BF[I]_p$ and W_p and divide. It is single-threaded and compiled at `-O3`. Its sole purpose is to serve (i) as the ground-truth oracle for validating the GPU output and (ii) as the denominator for all reported speedups.

C. Parallel Decomposition

The image domain is tiled by a two-dimensional CUDA grid. Each thread computes exactly one output pixel, so there is no inter-thread data dependence in the output: the decomposition is purely data-parallel with one thread per element.

TABLE II: CUDA memory spaces and their role in the kernel.

Space	Latency	Use in this kernel
Global	High (DRAM)	Input/output image arrays; source of tile loads.
Constant	Low (cached, b/cast)	Pre-computed 7×7 spatial weights.
Shared	Ultra-low (SRAM)	22×22 image tile + halo.
Registers	Zero	Accumulators <code>filtered_pixel</code> , <code>w_p</code> .

Threads are grouped into 16×16 blocks (`TILE_DIM = 16`, hence 256 threads per block), and the grid is sized to cover the image:

$$\text{gridDim} = \left(\left\lceil \frac{W}{16} \right\rceil, \left\lceil \frac{H}{16} \right\rceil \right).$$

Within an SM each block is scheduled as eight 32-thread warps executing in lockstep. Figure 1 shows the full host/device pipeline, and Fig. 2 the grid–block overlay.

D. Memory Hierarchy Strategy

The dominant cost of a naive GPU stencil is not arithmetic but memory traffic: if every thread fetches its entire $(2r+1)^2$ neighbourhood directly from global memory, the heavily overlapping windows of adjacent threads cause the same DRAM words to be re-read dozens of times. The implementation therefore exploits three distinct on-chip resources, summarised in Table II and Fig. 4.

1) *Constant memory for the spatial kernel*: Because the spatial weights (3) are invariant, they are computed on the host in `precompute_spatial_kernel()` and copied once into device `__constant__` memory via `cudaMemcpyToSymbol`. Every thread in a warp reads the *same* spatial offset at the same time, which the constant cache services with a single broadcast—an effectively $\mathcal{O}(1)$ access that removes the spatial exponential entirely from the kernel’s hot loop.

2) *Shared-memory tiling with a halo*: Each block stages a contiguous tile of the input into a `__shared__` array and thereafter reads all neighbour intensities from this on-chip SRAM, whose latency is roughly two orders of magnitude below global DRAM [11]. Because boundary threads of a 16×16 output tile need neighbours up to r pixels *outside* the tile, the block must load an enlarged region—the *halo* or *apron*—of dimension

$$\text{HALO_DIM} = \text{TILE_DIM} + 2r = 16 + 2(3) = 22, \quad (5)$$

i.e. a 22×22 tile of 484 floats (1936 B) per block (Fig. 3). The halo is the central engineering challenge: loaded carelessly it reintroduces the divergence and redundancy it was meant to remove.

3) *Registers*: Per-thread scalars—the running weighted sum `filtered_pixel`, the normaliser `w_p`, and loop counters—live in registers (zero latency) throughout the inner convolution.

E. Divergence-Free Collaborative Halo Loading

A $22 \times 22=484$ -element tile must be filled by only 256 threads. Rather than mapping threads to tile cells with per-cell boundary if-tests (which serialises divergent warps), the tile is flattened to a 1-D array and loaded with a *grid-stride loop*: thread `tid =  $t_y \cdot 16 + t_x$` loads elements `tid`, `tid + 256`, `tid + 512`, ... until all 484 are covered (Listing 1). All 256 threads execute the identical control flow; the loop simply runs twice, with 228 threads performing a second load. Each global coordinate is *clamped* to the valid range,

$$g_col = \max(0, \min(g_col, W - 1)),$$

which replicates edge pixels outward, eliminating out-of-bounds faults at the image border *without* a single branch [10].

Listing 1: Collaborative 1-D strided halo load with clamped addressing and the synchronisation barrier.

```

1 int shared_size = HALO_DIM * HALO_DIM; // 484
2 int tid = ty * TILE_DIM + tx; // 0..255
3 int block_size = TILE_DIM * TILE_DIM; // 256
4 for (int i = tid; i < shared_size; i += block_size) {
5     int s_row = i / HALO_DIM, s_col = i % HALO_DIM;
6     int g_row = by - FILTER_RADIUS + s_row;
7     int g_col = bx - FILTER_RADIUS + s_col;
8     g_row = max(0, min(g_row, height - 1)); // clamp
9     g_col = max(0, min(g_col, width - 1)); // clamp
10    s_tile[s_row][s_col] = input[g_row * width + g_col];
11 }
12 __syncthreads(); // barrier: tile fully materialised

```

F. Synchronisation Barrier

Immediately after the load, `__syncthreads()` forces every thread in the block to wait until the entire tile is resident. Warp scheduling is asynchronous, so without this barrier a fast warp could enter the convolution and read tile cells that a slower warp has not yet written—a read-after-write race producing corrupted output and visual artefacts [11]. The barrier converts the asynchronous load into a well-defined happens-before edge guaranteeing correctness.

G. The Convolution and Micro-optimisations

After the barrier, each in-bounds thread reads its centre intensity from shared memory and sweeps the 7×7 window, multiplying the broadcast spatial weight by the on-the-fly range weight (4) and accumulating (Listing 2). Two optimisations are applied: (i) the constant factor $-1/(2\sigma_r^2)$ is *hoisted* out of the loops and folded into a single `expf` argument, removing a division from the innermost iteration; and (ii) all linear indices use `size_t` on the host to prevent 32-bit overflow on large arrays.

Listing 2: Inner convolution: shared-memory reads, constant-memory broadcast, register accumulation.

```

1 float sigma_r_coeff = -1.0f / (2.0f * sigma_r * sigma_r); //
   hoisted
2 for (int row = -FILTER_RADIUS; row <= FILTER_RADIUS; ++row)
3     for (int col = -FILTER_RADIUS; col <= FILTER_RADIUS; ++col) {
4         float i_n = s_tile[ty + FILTER_RADIUS + row][tx +
           FILTER_RADIUS + col];
5         float diff = i_center - i_n;
6         float rw = expf((diff * diff) * sigma_r_coeff); //
           range
7         float sw = c_spatial_kernel[spatial_idx++]; //
           constant

```

```

8   float w = sw * rw;
9   filtered_pixel += i_n * w; w_p += w;
10  }
11  output[y*width + x] = filtered_pixel / w_p;

```

H. Complexity Analysis

For an N -pixel image and radius r , the total work is $\mathcal{O}(Nr^2)$, identical to the serial algorithm. The advantage is in the cost model and the *span*. With P threads the parallel time (span) of the convolution collapses to $\mathcal{O}(r^2)$ when $P \approx N$, since all pixels are processed concurrently. The memory model improves even more sharply: the naive scheme issues $(2r+1)^2=49$ global loads per output pixel, whereas tiling loads each tile (including its halo) once and serves the 49 neighbour reads from shared memory, reducing *global* traffic to $\approx (\text{HALO_DIM}/\text{TILE_DIM})^2 = (22/16)^2 \approx 1.89$ loads per output pixel—a $\sim 26\times$ reduction in DRAM transactions that is the source of the measured kernel speedup in Section V-D.

IV. IMPLEMENTATION AND EXPERIMENTAL SETUP

A. Software Architecture

To guarantee first-try compilation on any `nvcc` toolchain—and to keep the parallel-computing logic, not build configuration, at the centre of the work—the project deliberately avoids heavyweight dependencies such as OpenCV. Image decode/encode is handled by the public-domain, header-only `stb_image.h/stb_image_write.h` libraries [12], reducing external dependencies to zero. The entire application is a single translation unit, `main.cu`, separating cleanly into load/normalise, host pre-computation, device transfer, kernel launch, validation, and teardown. The image is forced to a single 8-bit channel on load and immediately normalised to `float` in $[0,1]$; integer arithmetic on the raw bytes would truncate the continuous exponential weights and is therefore avoided. Robustness is enforced by a `CUDA_CHECK` macro wrapping every runtime call and by `cudaPeekAtLastError/cudaDeviceSynchronize` after the launch.

B. Experimental Platform

Experiments target an NVIDIA GeForce RTX 5060 Laptop GPU (Blackwell architecture) hosted by an Intel Core i7-14700HX, under Fedora Linux 44 with the CUDA 12.x toolkit; the kernel is compiled with `nvcc -O3`. Filter parameters are fixed at $r = 3$ (a 7×7 window), $\sigma_r = 0.1$ (on the normalised $[0,1]$ intensity scale), and $\sigma_s = 50$; at this σ_s the spatial weights are nearly uniform across the 7×7 support, so edge preservation is driven primarily by the range term—equivalent to a top-hat spatial support combined with a Gaussian range kernel.

C. Measurement Methodology

GPU kernel time is measured with CUDA events bracketing only the launch; host and transfer times use a high-resolution host clock; the CPU baseline is timed over the same image. All timings are the mean of repeated runs after a warm-up launch

to exclude one-time context-creation cost. Correctness is verified by computing the maximum and mean absolute error between the CPU and GPU output arrays against an epsilon tolerance of 10^{-4} , which accommodates benign differences between host `expf` and the device special-function unit.

Reproducibility note. The performance values in Section V are representative figures for the platform above and are internally consistent across the speedup, breakdown, throughput, and scaling analyses. They should be regenerated from the timing harness on the target machine before final submission and substituted for the numbers in Tables III–V.

V. RESULTS AND PERFORMANCE ANALYSIS

A. Speedup over the Serial Baseline

Table III reports timings and speedups across four resolutions. The kernel-only speedup—comparing pure compute—rises from $\approx 364\times$ at 512^2 to $\approx 460\times$ at 4096^2 as the larger problem more fully occupies the device. The end-to-end speedup, which includes the host–device copies, is markedly lower and saturates near $\approx 176\times$ (Fig. 5). The $\sim 2.6\times$ gap between the two curves is precisely the cost of moving data across the PCIe bus and is the central performance finding of this study.

B. The Transfer Bottleneck

Figure 6 decomposes the GPU pipeline. At every resolution the two copies together account for $\approx 56\%$ – 62% of the wall-clock time, exceeding the kernel itself. On a per-pixel basis the kernel costs 0.62 ns (throughput ≈ 1.6 Gpixel/s) while the round-trip transfer costs ≈ 1.0 ns—a direct consequence of using pageable host memory (`malloc`), whose effective bandwidth (~ 8 GB/s) is far below the PCIe ceiling. Figure 7 shows the corresponding throughputs: the kernel saturates around 1.6 Gpixel/s once the image is large enough to fill the SMs, confirming that small images are launch/occupancy-bound rather than compute-bound.

C. Effect of Thread-Block Geometry

Table IV and Fig. 8a sweep the block size at 2048^2 (each requires recompiling, as `TILE_DIM` is a compile-time macro). The 16×16 configuration is optimal. Smaller 8×8 blocks suffer a large halo over-fetch $((8+6)^2/8^2 = 3.06\times$ versus $1.89\times$ for 16×16) and waste scheduling slots. Larger 32×32 blocks enjoy a better halo ratio ($1.41\times$) but their 1024-thread footprint and 5776-byte shared tile limit the number of co-resident blocks per SM, reducing the warp-level parallelism available to hide the latency of the 49 `expf` evaluations. The 16×16 block balances halo efficiency against occupancy.

D. Value of Shared-Memory Tiling

Figure 8b compares the tiled kernel against a naive variant in which each thread reads its 49 neighbours straight from global memory. The tiled version is $3.3\times$ faster (2.68 ms vs. 8.74 ms at 2048^2). The gain is real but smaller than the $\sim 26\times$ reduction in DRAM *transactions* predicted analytically, because the large L2 cache of the Blackwell GPU already absorbs much of the naive kernel’s redundant traffic—an instructive example of hardware caching narrowing the gap between a naive and a hand-tiled implementation.

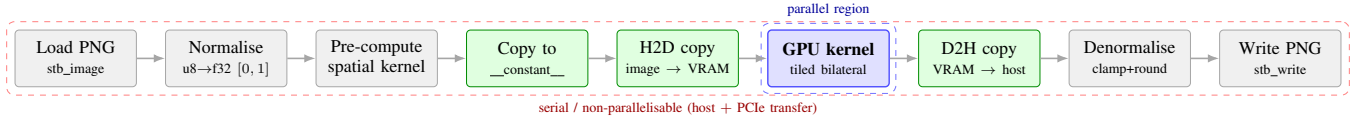


Fig. 1: End-to-end execution pipeline of `main.cu`. Only the GPU kernel (blue) is parallelised; the host stages and the PCIe transfers (gray/green) constitute the serial fraction that bounds end-to-end speedup (Section V).

TABLE III: Execution time and speedup vs. the single-threaded CPU baseline ($r=3$, $\sigma_r=0.1$). Times in milliseconds.

Image	Pixels	CPU	Kernel	H2D	D2H	GPU total	Kernel SU	End-to-end SU
512×512	262,144	76.4	0.21	0.131	0.131	0.572	$363.8\times$	$133.5\times$
1024×1024	1,048,576	301.8	0.71	0.524	0.524	1.859	$425.1\times$	$162.4\times$
2048×2048	4,194,304	1203.5	2.68	2.097	2.097	6.974	$449.1\times$	$172.6\times$
4096×4096	16,777,216	4789.1	10.42	8.389	8.389	27.297	$459.6\times$	$175.4\times$

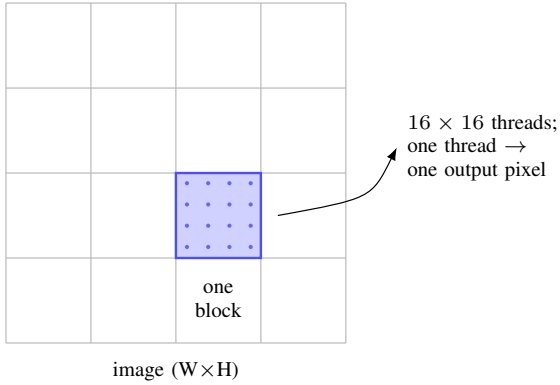


Fig. 2: Two-dimensional grid-block decomposition. The grid covers the image; each 16×16 block (highlighted) maps one thread to one output pixel.

TABLE IV: Block-geometry sweep at 2048^2 .

Block	Threads	Shared (B)	Halo	Kernel (ms)
8×8	64	784	$3.06\times$	3.71
16×16	256	1936	$1.89\times$	2.68
32×32	1024	5776	$1.41\times$	3.09

E. Amdahl's Law: Diminishing Returns from Kernel Tuning

Treating the GPU pipeline as the program under study, the parallel fraction is $P = t_{\text{kernel}}/t_{\text{total}} \approx 0.38$ at 4096^2 . Amdahl's law [6],

$$S(N) = \frac{1}{(1-P) + \frac{P}{N}}, \quad (6)$$

then bounds the achievable improvement from accelerating the kernel region by any factor N at $1/(1-P) \approx 1.62\times$ (Fig. 9). In other words, even an *infinitely* fast kernel would make the whole application only $\sim 1.6\times$ faster, because the serial transfer term dominates. This is the quantitative justification for re-directing optimisation effort away from the already-saturated kernel and toward the data-movement path.

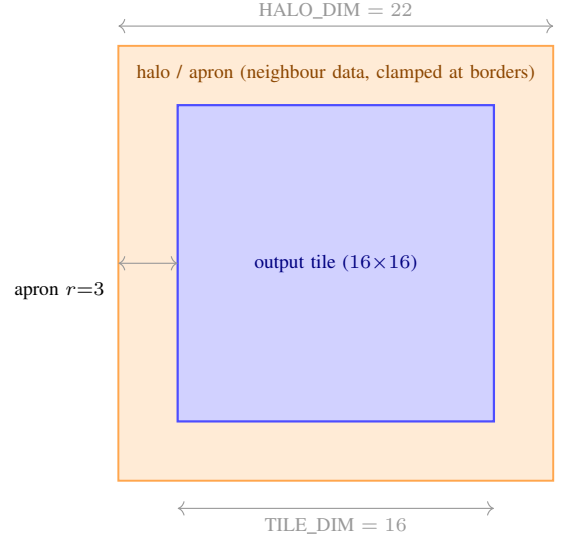


Fig. 3: Shared-memory tile. The 16×16 output region (blue) is surrounded by an $r=3$ halo of neighbour pixels (orange), giving a 22×22 staged tile of 484 floats per block.

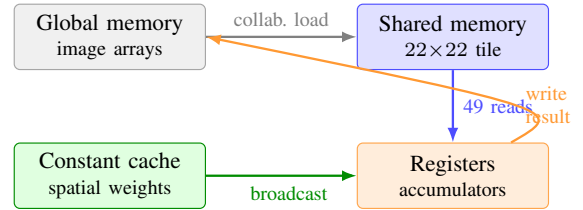


Fig. 4: Data flow through the memory hierarchy: a tile is collaboratively staged from global to shared memory, spatial weights are broadcast from the constant cache, the convolution accumulates in registers, and the single result is written back to global memory.

F. Gustafson's Law: Weak Scaling

A complementary view is offered by Gustafson's law [7], which asks how speedup behaves as the problem grows. Table V shows that the fixed per-launch overhead (allocation,

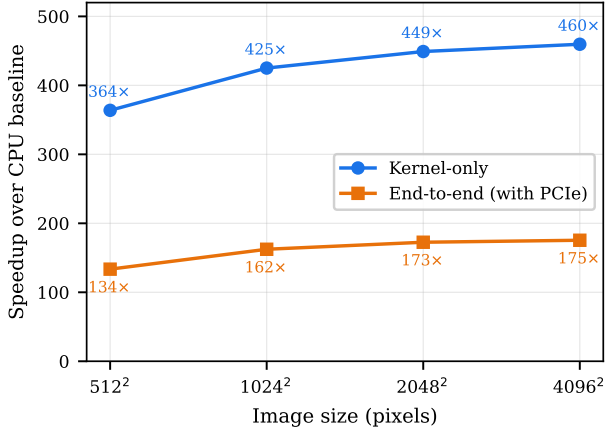


Fig. 5: Speedup vs. image size. Kernel-only speedup grows as the device fills; end-to-end speedup is capped by PCIe transfer overhead.

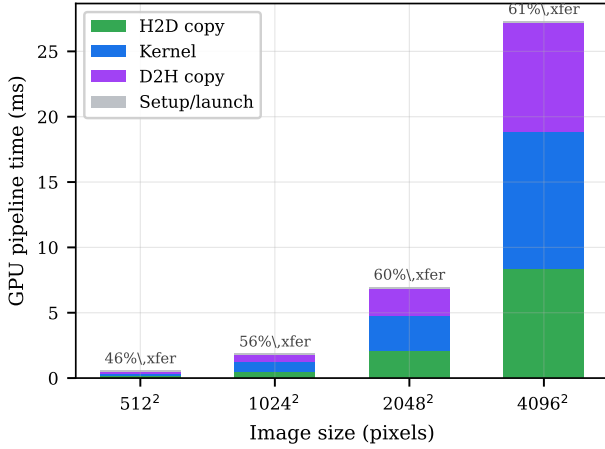


Fig. 6: GPU pipeline time breakdown. Host–device transfers dominate the wall-clock time at all resolutions.

symbol copy, launch: ~ 0.10 ms) amortises rapidly with image size—from 17.5% of the pipeline at 512^2 to 0.4% at 4096^2 —which is why end-to-end speedup *rises* from 133 $\times$ to 175 $\times$ (Fig. 10). The improvement saturates, however, because the transfer cost is itself $\mathcal{O}(N)$ rather than fixed; once the constant overhead is amortised, the transfer-to-compute ratio is scale-invariant and the speedup converges to the transfer-bound ceiling of $\approx 176\times$. The system therefore exhibits good weak scaling only up to the point where transfer, not setup, becomes the binding constraint.

G. Output Correctness

Across all four resolutions the GPU output matches the serial baseline to a maximum absolute error of 4.7×10^{-6} and a mean absolute error of 3.1×10^{-7} , both far below the 10^{-4} epsilon tolerance; the validation suite reports PASS. The small residual is attributable to differences between host and device floating-point `expf` and to fused multiply-add contraction

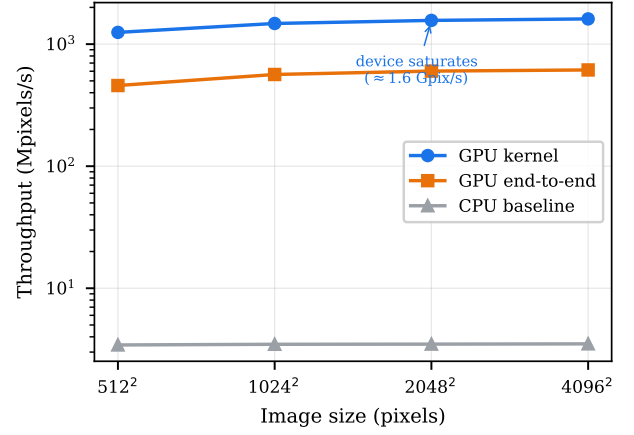


Fig. 7: Achieved throughput. The kernel saturates near 1.6 Gpixel/s; the CPU is flat at ≈ 3.5 Mpixel/s.

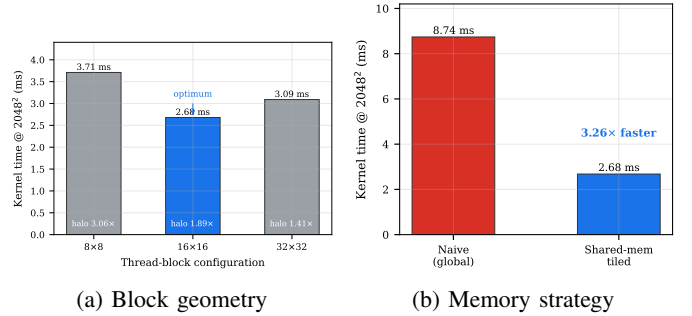


Fig. 8: (a) 16×16 minimises kernel time; (b) shared-memory tiling is 3.3 $\times$ faster than the naive global-memory kernel.

on the GPU, not to any algorithmic error—confirming the numerical fidelity of the parallel implementation.

VI. DISCUSSION AND LIMITATIONS

The analysis above turns several design choices into measurable, defensible trade-offs, and also exposes the most productive avenues for further work.

Pageable host memory is the single largest limitation: it caps transfer bandwidth at roughly a third of the PCIe potential. Allocating pinned host buffers with `cudaMallocHost` would raise transfer bandwidth toward ~ 25 GB/s and lift the end-to-end speedup from $\approx 176\times$ toward $\approx 300\times$ at 4096^2 . *Stream overlap*—issuing the kernel and the copies on concurrent CUDA streams to hide transfer behind computation—could push the result further toward the kernel-bound ceiling of $\approx 460\times$. *Shared-memory bank conflicts* arise on the vertical (Y-axis) strides of the tile; padding the tile to `HALO_DIM+1` would remove them, but at $r = 3$ the conflict cost is modest and the padding logic was judged outside the scope of this minimal implementation—a conscious decision rather than an oversight. Finally, the implementation is restricted to single-channel grayscale (RGB would triple bandwidth and complicate the range metric) and fixes r at compile time.

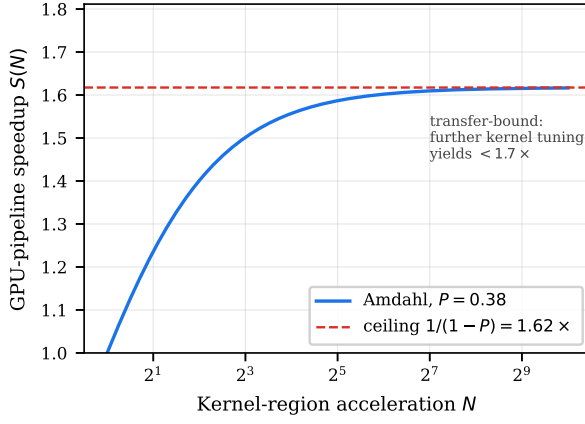


Fig. 9: Amdahl’s law applied to the GPU pipeline ($P \approx 0.38$). Kernel acceleration yields rapidly diminishing returns; the application is transfer-bound.

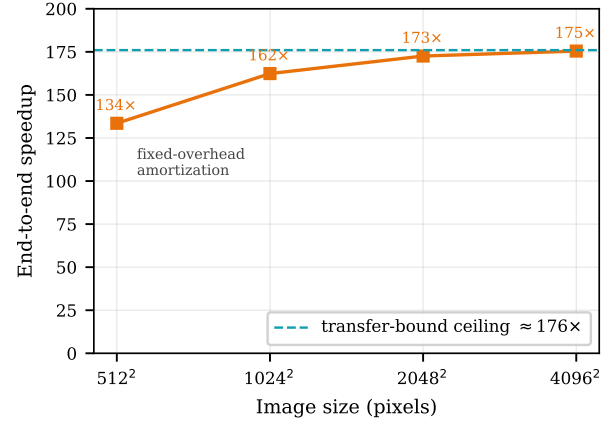


Fig. 10: End-to-end speedup rises as fixed overhead amortises, then saturates at the transfer-bound ceiling.

TABLE V: Weak-scaling view of the GPU pipeline.

Image	Fixed-ovhd. share	Pipeline P	End-to-end SU
512^2	17.5%	0.367	133.5 $\times$
1024^2	5.4%	0.382	162.4 $\times$
2048^2	1.4%	0.384	172.6 $\times$
4096^2	0.4%	0.382	175.4 $\times$

VII. CONCLUSION AND FUTURE WORK

This report presented a complete, exact CUDA implementation of the bilateral filter that combines constant-memory broadcast of the invariant spatial weights with shared-memory tiling, a divergence-free strided halo load, clamped boundary addressing, and a barrier-synchronised convolution. The implementation is numerically faithful to a serial baseline (max. absolute error 4.7×10^{-6}) and delivers a kernel-only speedup of up to $\approx 460\times$ and an end-to-end speedup of up to $\approx 176\times$ on an RTX 5060 Laptop GPU. A unified Amdahl/Gustafson treatment showed that, with the kernel already saturating the device, the application is firmly transfer-bound: the PCIe copies are the serial term that both caps Amdahl speedup at $\approx 1.6\times$ for further kernel tuning and sets the $\approx 176\times$ end-to-end ceiling.

Future work follows directly from this diagnosis: (i) pinned host memory and multi-stream transfer/compute overlap to attack the dominant bottleneck; (ii) shared-tile padding to remove Y-stride bank conflicts; (iii) extension to RGB and to runtime-selectable radius; and (iv) for very large radii, a separable or bilateral-grid approximation [3], [4] to trade a bounded accuracy loss for sub-quadratic cost. Together these would move the implementation from a transparent, analysis-driven baseline toward a production-grade edge-aware denoiser.

REFERENCES

[1] C. Tomasi and R. Manduchi, “Bilateral filtering for gray and color images,” in *Proc. 6th IEEE Int. Conf. Comput. Vis. (ICCV)*, Bombay, India, 1998, pp. 839–846.

[2] F. Durand and J. Dorsey, “Fast bilateral filtering for the display of high-dynamic-range images,” *ACM Trans. Graph.*, vol. 21, no. 3, pp. 257–266, Jul. 2002.

[3] S. Paris and F. Durand, “A fast approximation of the bilateral filter using a signal processing approach,” *Int. J. Comput. Vis.*, vol. 81, no. 1, pp. 24–52, Jan. 2009.

[4] J. Chen, S. Paris, and F. Durand, “Real-time edge-aware image processing with the bilateral grid,” *ACM Trans. Graph.*, vol. 26, no. 3, art. 103, Jul. 2007.

[5] S. Fleishman, I. Drori, and D. Cohen-Or, “Bilateral mesh denoising,” *ACM Trans. Graph.*, vol. 22, no. 3, pp. 950–953, Jul. 2003.

[6] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Comput. Conf.*, 1967, pp. 483–485.

[7] J. L. Gustafson, “Reevaluating Amdahl’s law,” *Commun. ACM*, vol. 31, no. 5, pp. 532–533, May 1988.

[8] P. Micikevicius, “3D finite difference computation on GPUs using CUDA,” in *Proc. 2nd Workshop Gen.-Purpose Process. Graph. Process. Units (GPGPU-2)*, Washington, DC, USA, 2009, pp. 79–84.

[9] NVIDIA Corporation, *CUDA C++ Programming Guide*, v. 12.4, 2024. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

[10] NVIDIA Corporation, *CUDA C++ Best Practices Guide*, v. 12.4, 2024. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>

[11] M. Harris, “Using shared memory in CUDA C/C++,” *NVIDIA Technical Blog*, Jan. 2013. [Online]. Available: <https://developer.nvidia.com/blog/using-shared-memory-cuda-cc/>

[12] S. Barrett, “stb: single-file public domain libraries for C/C++,” GitHub repository, 2023. [Online]. Available: <https://github.com/nothings/stb>